LogInjector: Detecting Web Application Log Injection Vulnerabilities

A Dissertation Submitted in Partial Fulfillment of the Requirements for the Degree of Doctor of Philosophy in Computer Science

Candidate: [Your name]

Supervisor: [Supervisor name]

Institution: [University, Department]

Year: 2026

> **Note on this draft.** The text, design, formal model and methodology are complete. Every number in Chapter 7 is a bracketed placeholder such as [X]. Replace them with results from your own experiments before submission; nothing here reports a measurement that was actually run. Verify every reference against the original source and add citations required by your institution.

Abstract

Logs are the primary record of what a web application did, and the primary evidence after a security incident. Yet most applications write attacker-controlled data into logs with no neutralization. This class of flaw, catalogued as CWE-117 (Improper Output Neutralization for Logs), lets an attacker forge log entries, hide malicious activity, attack log viewers through stored cross-site scripting, and in the worst case trigger code execution inside the logging framework itself, as the Log4Shell vulnerability (CVE-2021-44228) showed in December 2021.

Despite this, log injection receives little attention from automated analysis tools. General taint analyzers treat logging calls as benign or flag them with high false-positive rates, because they cannot tell a dangerous log write from a safe one. They ignore the log format, the framework's interpolation features, and the downstream consumers that give an injected payload its meaning.

This dissertation presents **LogInjector**, a hybrid static-dynamic system for detecting log injection vulnerabilities in web applications. LogInjector makes four contributions. First, a taxonomy and formal model of log injection that defines a vulnerability as a tainted flow into a log sink whose *rendering context* lets the payload change the structure or semantics of the log. Second, a sink-and-context model for widely used logging frameworks (Log4j 2, Logback/SLF4J, java.util.logging, Python logging, Winston and Pino for Node.js, Monolog for PHP), covering format strings, lookups, structured encoders and layout patterns. Third, a context-sensitive static taint analysis that tracks web inputs to log sinks and recognizes framework-specific sanitizers. Fourth, a dynamic confirmation stage that replays candidate flows with context-aware payloads and checks the resulting log artifacts with format-specific oracles, reducing false positives.

We evaluate LogInjector on [N] open-source web applications and a benchmark of [M] seeded vulnerabilities. [Summary of results to be inserted: precision, recall, comparison against baseline tools, number of new vulnerabilities found and disclosed.]

Keywords: log injection, CWE-117, log forging, taint analysis, web application security, static analysis, dynamic analysis, Log4Shell, security testing

Chapter 1 — Introduction

1.1 Motivation

Every production web application logs. Logs feed incident response, fraud detection, compliance audits, billing, and security information and event management (SIEM) pipelines. Developers log generously because logging is cheap and debugging without logs is expensive. The values they log most often are exactly the ones an attacker controls: usernames, request paths, headers such as `User-Agent` and `X-Forwarded-For`, query parameters, and error messages that echo input.

Logging is usually treated as a passive, write-only side channel. That assumption is wrong. A log line is a structured artifact with a format: newline-delimited text, JSON, key-value pairs, CSV, syslog, or HTML in a log viewer. When attacker data enters that artifact without neutralization, the attacker can change its structure. Inserting `\r\n` forges a second, fake entry. Inserting HTML attacks the administrator who views the log in a browser. Inserting a formula such as `=HYPERLINK(...)` attacks the analyst who exports logs to a spreadsheet. Inserting a framework lookup such as `${jndi:ldap://...}` caused remote code execution in Apache Log4j 2 versions before 2.15.0.

Log4Shell (CVE-2021-44228) demonstrated the scale of the problem. A single logging call on attacker input became one of the most widely exploited vulnerabilities on record, affecting a large share of Java services worldwide. It also showed that the danger of a log write depends on the *logging framework's semantics*, not only on the data flow.

1.2 Problem statement

Existing static analysis tools find injection flaws by tracking untrusted data (sources) to dangerous operations (sinks). For SQL injection or command injection the sink and its danger are clear. For logging they are not:

- **Sink ambiguity.** Thousands of logging calls exist in a typical codebase. Most log constants or internal state. Tools that flag every tainted log call produce reports developers ignore.
- **Context blindness.** Whether a payload is dangerous depends on the appender, the layout or encoder, the format string, and the framework version. A JSON encoder that escapes newlines makes CRLF forging impossible; a plain pattern layout does not.
- **Downstream semantics.** Some impact appears only when a consumer (a web log viewer, a SIEM parser, a spreadsheet) reads the log. Source-to-sink analysis inside the application does not model these consumers.

- **Sanitizer diversity.** Neutralization is done with ad hoc `replace` calls, framework options such as `%enc{...}{CRLF}`, or structured logging. Tools rarely recognize these.

The result is a gap: log injection is common, sometimes critical, and poorly served by automated detection.

1.3 Thesis statement

Log injection vulnerabilities in web applications can be detected with high precision by combining static taint analysis with an explicit model of each logging framework's rendering context, and by confirming candidate flows dynamically with format-aware oracles.

1.4 Research questions

1. **RQ1 (Prevalence).** How common are unneutralized flows from web inputs to log sinks in real-world web applications, and which injection classes do they enable?
2. **RQ2 (Effectiveness).** What precision and recall does LogInjector achieve, and how does it compare with general-purpose taint analyzers and security scanners?
3. **RQ3 (Context modeling).** How much do framework-context modeling and dynamic confirmation each contribute to precision? (Ablation study.)
4. **RQ4 (Cost).** What are LogInjector's analysis time and memory costs, and are they acceptable for continuous integration?
5. **RQ5 (Real-world impact).** Does LogInjector find previously unknown vulnerabilities that maintainers confirm?

1.5 Contributions

1. A **taxonomy** of log injection with seven classes, mapped to rendering contexts and downstream consumers (Chapter 2).
2. A **formal model** that defines a log injection vulnerability as a tainted, unneutralized flow into a sink whose context grammar admits structure-changing payloads (Chapter 4).
3. **LogInjector**, a hybrid detector combining a framework sink-and-context model, context-sensitive static taint analysis, and dynamic confirmation with format oracles (Chapters 5 and 6).
4. An **evaluation** on real applications and a seeded benchmark, released with the tool for reproducibility (Chapter 7).

1.6 Structure of the dissertation

Chapter 2 gives background and the taxonomy. Chapter 3 reviews related work. Chapter 4 states the threat model and formalization. Chapter 5 describes LogInjector’s design and Chapter 6 its implementation. Chapter 7 presents the evaluation. Chapter 8 discusses limitations, threats to validity and ethics. Chapter 9 concludes.

Chapter 2 — Background and Taxonomy

2.1 Logging in web applications

A modern logging pipeline has four stages. The **application** calls a logging API (`logger.info(...)`). The **framework** formats the event through a layout or encoder and passes it to an **appender** (file, console, socket, syslog, HTTP). A **transport and storage** layer (Filebeat, Fluentd, Logstash, a cloud logging service) ships and indexes it. **Consumers** read it: engineers in a terminal, administrators in a web dashboard, SIEM correlation rules, alerting systems, and analysts exporting to spreadsheets.

The framework layer matters most for injection. It decides whether the message is a format string, whether placeholders such as `{}` or `%s` are interpolated, whether lookups such as `${...}` are evaluated, and how special characters are escaped. Common frameworks studied in this work:

Ecosystem	Framework	Interpolation	Notable features
Java	Log4j 2	`{}` parameters, `${}` lookups	Lookups (JNDI removed by default from 2.15/2.16), `%enc{}{CRLF}`, JSON Template Layout
Java	SLF4J + Logback	`{}` parameters	Pattern layout, `%replace`, Logstash JSON encoder
Java	java.util.logging	`{0}` MessageFormat	SimpleFormatter, XMLFormatter
Python	logging	`%`-style args	Formatter, `extra` fields, JSON formatters via third-party packages
Node.js	Winston, Pino	`%s` (Winston splat), objects	JSON by default (Pino), custom formats
PHP	Monolog	`{placeholder}` context	LineFormatter, JsonFormatter, HtmlFormatter

Table 2.1 — Logging frameworks studied

2.2 Definition

We define **log injection** as a flaw in which data originating from an untrusted actor is written to a log in a form that lets the actor alter the log's structure, the meaning of other entries, or the behavior of the logging framework or a log consumer. This generalizes CWE-117, which focuses on output neutralization, and includes framework-level interpretation such as lookups.

2.3 Taxonomy

We classify log injection by the *context* in which the payload takes effect.

Class	Context	Example payload	Impact
C1 Log forging	Line-oriented text	`admin\r\n2026-11-02 INFO Login success user=root`	Fake entries, misleading forensics
C2 Log truncation and hiding	Text, terminals	NUL bytes, `\b`, ANSI escape sequences	Entries hidden or rewritten on display
C3 Structured-field injection	JSON, key-value, CSV, syslog	`","level":"INFO","user":"root` in naive string-built JSON	Corrupted fields, SIEM rule evasion
C4 Viewer injection (stored XSS)	HTML log dashboards	`<script>... </script>` in `User-Agent`	Session theft of administrators
C5 Formula injection	Spreadsheet export	`=HYPERLINK("http://evil","x")`	Data exfiltration, phishing of analysts
C6 Framework interpretation	Lookups and format strings	`${jndi:ldap://host/a}`, `${env:SECRET}`	Remote code execution, secret disclosure
C7 Resource abuse	Any appender	Very large or high-rate payloads	Disk exhaustion, log rotation evicting evidence

Table 2.2 — Taxonomy of log injection

Classes C1–C3 attack *integrity* of the record; C4 and C5 attack *consumers*; C6 attacks the *application* itself; C7 attacks *availability*. LogInjector targets C1–C6. C7 is a rate and volume problem better handled by quota controls, and we discuss it only in Chapter 8.

2.4 Neutralization strategies

Correct neutralization depends on context:

- **Encoding at the layout.** Log4j 2's `%enc{%m}{CRLF}` and Logback's `%replace` escape control characters for every message, independent of call sites.
- **Structured logging.** A proper JSON encoder escapes quotes and control characters, removing C1 and C3, but not C4 if the viewer renders fields as HTML.
- **Call-site sanitization.** Code such as `input.replaceAll("[\r\n]", "_")`. Effective for C1 only, and only if every path is covered.
- **Framework configuration.** Disabling message lookups (`log4j2.formatMsgNoLookups`) or upgrading past vulnerable versions removes C6.
- **Consumer-side encoding.** Log viewers that HTML-encode on output remove C4 regardless of the log content.

This context dependence is the central insight of the dissertation: a sanitizer is only meaningful relative to a context, so a detector must model both.

Chapter 3 — Related Work

LogInjector sits at the intersection of four research lines: static taint analysis for web applications, context-sensitive sanitization, dynamic and black-box injection testing, and log analysis. None of them treats log injection as a first-class, context-dependent vulnerability class.

3.1 Static taint analysis for web applications

Livshits and Lam [1] applied pointer-analysis-based taint tracking to Java web applications and found SQL injection, cross-site scripting and HTTP response splitting. Pixy [2] brought flow-sensitive, interprocedural taint analysis to PHP. TAJ [3] scaled taint analysis to large Java applications with hybrid thin slicing and modeling of frameworks such as Struts. FlowDroid [4] showed that precise context-, flow- and object-sensitive analysis with lifecycle modeling is feasible for Android. Spoto et al. [5] applied abstract interpretation to injection detection in Java. Code property graphs [6] and query-based engines such as CodeQL [7] and Semgrep let analysts express taint queries declaratively.

These systems model logging, if at all, as a generic sink or as a benign call. CodeQL and Semgrep ship log-injection queries that flag tainted arguments to logging methods, but they do not reason about the framework’s layout, encoder or version. The consequence is predictable: either every log call on user data is reported, or none is. LogInjector reuses taint-analysis machinery but adds an explicit rendering-context model.

3.2 Context-sensitive sanitization

Saner [8] combined static and dynamic analysis to check whether sanitizers in PHP applications actually neutralize attacks, by modeling string operations with automata and then testing them. Samuel, Saxena and Song [9] and ScriptGard [10] showed that sanitizer correctness depends on the output context (HTML body, attribute, URL, JavaScript) and proposed context-sensitive auto-sanitization. Su and Wassermann [11] formalized injection as an attack that changes the syntactic structure of an output language.

LogInjector adopts this view directly. A log line is an output language with its own grammar, and the grammar depends on the layout. A sanitizer that removes `\n` is sound for a newline-delimited text layout but irrelevant for an HTML viewer. Our formal model (Chapter 4) extends the Su–Wassermann structural definition to log formats and framework interpretation.

3.3 Dynamic and black-box injection testing

Black-box scanners send payloads and observe responses. Doupé, Cova and Vigna [12] and Bau et al. [13] found that such scanners miss many vulnerability classes, especially stored and second-order ones. Log injection is the extreme case: the effect lands in a file the scanner cannot see. Kieyzun et al. [14] generated concrete SQL injection and XSS exploits from taint-guided concolic execution. Schwartz, Avgerinos and Brumley [15] surveyed dynamic taint analysis and its pitfalls, notably over- and under-tainting.

LogInjector’s confirmation stage is grey-box. It knows the candidate flow and the sink context from static analysis, and it has access to the produced log artifacts, so it can test precisely what a black-box scanner cannot observe.

3.4 Log analysis and log security

Log parsing research such as Drain [16] and the Loghub benchmark [17] studies how to turn raw logs into structured events for anomaly detection. This work assumes logs are trustworthy. Injected entries poison the input to these parsers and to SIEM rules built on them, which motivates detection at the source.

Practitioner guidance covers log injection well: MITRE CWE-117 [18], the OWASP Logging Cheat Sheet [19], and the Apache Log4j security advisories for CVE-2021-44228 and related issues [20]. Post-Log4Shell, much industry effort went into detecting vulnerable *versions* through software composition analysis. Version scanning answers whether a dangerous framework is present; it does not answer whether attacker data reaches it, or whether other injection classes exist.

3.5 Gap

Approach	Tracks web inputs to log calls	Models framework context	Models downstream consumers	Confirms dynamically
Generic taint analyzers (CodeQL, Semgrep queries)	Yes	No	No	No
Context-sensitive sanitization research	Partly	Yes, for HTML/JS	No	Partly (Saner)
Black-box scanners	No (indirect)	No	No	Yes, but cannot observe logs
Software composition analysis	No	Version only	No	No
LogInjector	**Yes**	**Yes**	**Yes**	**Yes**

Table 3.1 — Comparison with existing approaches

Chapter 4 — Threat Model and Formalization

4.1 Threat model

Attacker. A remote, unauthenticated or low-privilege user who can send arbitrary HTTP requests to the application. The attacker controls request lines, headers, cookies, bodies, uploaded file names, and any data the application later reads back from stores the attacker can write to (second-order input).

Goal. One or more of: (a) forge or hide log entries to mislead investigators; (b) evade detection rules; (c) execute script in an administrator's browser through a log viewer; (d) attack analysts through exported logs; (e) cause the logging framework to perform unintended actions such as network lookups or secret disclosure.

Assumptions. The attacker cannot read or write log files directly, cannot change logging configuration, and does not have code execution before the attack. The application's source or bytecode and its logging configuration are available to the analyst running LogInjector.

Out of scope. Insider tampering with log storage, attacks on log transport encryption, and denial of service through volume (class C7).

4.2 Formal model

We model a program P as an interprocedural control-flow graph with a set of **sources** S (statements reading attacker input) and **log sinks** K (calls into a logging API). A log sink k has a **rendering context** ctx(k), determined by the framework F, its version v, the logger's configuration (layout or encoder L and appender A), and the argument position of the tainted value (message template, parameter, or structured field).

Each context c defines an **output grammar** G_c describing well-formed log artifacts: for a newline-delimited pattern layout, one record per line with a fixed field order; for a JSON encoder, one JSON object per record; for an HTML viewer, an HTML document. Following Su and Wassermann [11], let a log record r be produced from a template with holes filled by values. Let t be the substring of r contributed by tainted data.

Definition 1 (Structural log injection). A record r with tainted substring t is *structurally injected* in context c if the parse tree of r under G_c contains a node that is not fully covered by t but whose boundaries fall inside t. Informally, t changes the structure of the record rather than only filling a leaf value.

Definition 2 (Interpretive log injection). A sink k with framework F is *interpretively injectable* if F applies an interpretation function I_F (lookups, format-string expansion) to t before rendering, and I_F has observable effects beyond string substitution, such as network access, environment access or recursive expansion.

Definition 3 (Neutralizer). A function n is a neutralizer for context c if, for every string x, inserting n(x) into any hole yields a record in which n(x) is confined to a single leaf of the parse tree under G_c, and I_F is not applied to it.

Definition 4 (Vulnerability). A log injection vulnerability is a feasible path from a source s in S to a sink k in K along which the tainted value is not passed through a neutralizer for ctx(k), and either Definition 1 or Definition 2 holds for some attacker-chosen input.

This definition has two useful consequences. First, a sanitizer that strips CR and LF is a neutralizer for the plain-text context but not for the HTML-viewer context, so the same flow can be safe or vulnerable depending on the consumer. Second, a JSON encoder that escapes all control characters acts as a neutralizer for C1 and C3 *at the layout level*, so call sites that use it need no further sanitization for those classes.

4.3 Context lattice

Contexts form a partial order by strictness. A neutralizer for a stricter context (one whose grammar has more structure-sensitive characters) is also a neutralizer for a laxer one when the laxer grammar’s special characters are a subset. LogInjector uses this order to summarize a sanitizer once and reuse the summary across sinks: a call-site function that HTML-encodes and removes control characters is recorded as neutralizing {text, kv, html}, but not {json} unless it also escapes quotes and backslashes.

Chapter 5 — Design of LogInjector

LogInjector runs in four stages: configuration analysis, sink-and-context resolution, static taint analysis, and dynamic confirmation. Static stages produce candidate flows annotated with a context; the dynamic stage confirms or rejects each candidate and produces a proof-of-concept.

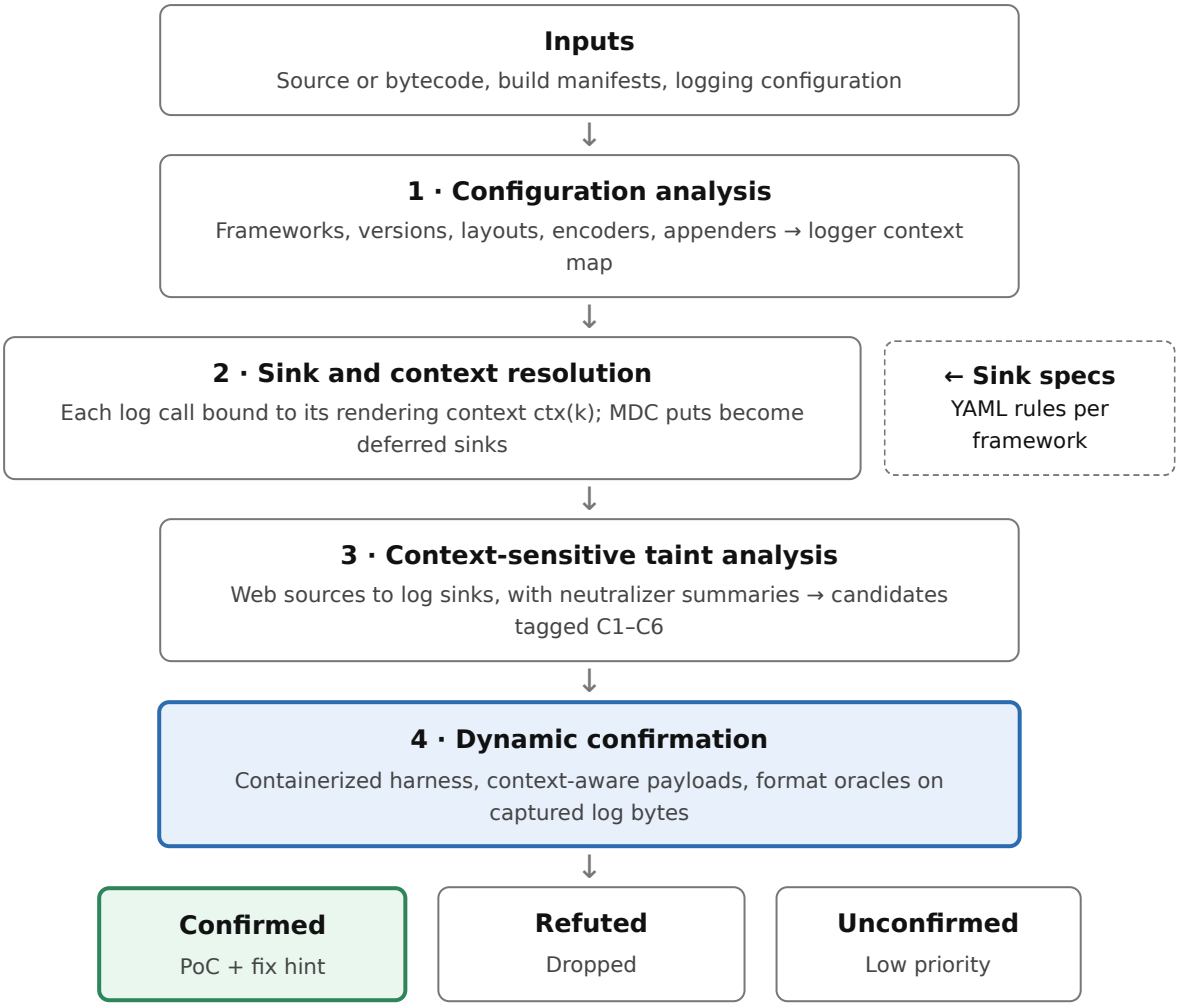


Figure 5.1 — LogInjector architecture. Stages 1–3 over-approximate; Stage 4 sorts each candidate into confirmed, refuted or unconfirmed.

5.1 Stage 1 — Configuration and dependency analysis

LogInjector first determines which logging frameworks and versions the application uses, from build manifests (Maven `pom.xml`, Gradle files, `requirements.txt`, `package.json`, `composer.json`) and bundled libraries. It then parses logging configuration files (`log4j2.xml`, `logback.xml`, `logging.ini` or `dictConfig` dictionaries, Winston and Pino setup code, Monolog handler setup) to build a map from each logger name to its layouts, encoders and appenders.

The output is a **logger context map**: for each logger name pattern, the set of output contexts it can produce (for example, `com.app.auth → {text-pattern, json}`), the layout-level neutralizers in effect (such as `%enc{%m}{CRLF}`), and whether interpretation such as message lookups is enabled for that framework version and configuration. Where configuration is set programmatically and cannot be resolved statically, LogInjector conservatively assumes the framework default.

5.2 Stage 2 — Sink and context resolution

LogInjector ships a **sink specification** for each supported framework. Each entry names a method, the argument roles (template, parameter, throwable, structured field, MDC or context map), and the interpretation rules for each role. For example, in SLF4J `logger.info(String format, Object... args)`, the template is parsed for `{}` placeholders and parameters are substituted as strings; in Log4j 2 before 2.15.0, lookups could be evaluated in the formatted message by default; in Python's `logging`, `%`-style arguments are substituted lazily; Monolog interpolates `{name}` placeholders from the context array only when the PsrLogMessageProcessor is installed.

For each call site, LogInjector resolves the logger instance (through field initializers such as `LoggerFactory.getLogger(Foo.class)` and dependency injection) to a logger name, looks it up in the context map, and attaches ctx(k) to the sink. MDC and thread-context puts are modeled as deferred sinks that fire wherever a layout references them (`%X{user}`).

5.3 Stage 3 — Context-sensitive static taint analysis

The taint engine tracks data from sources to the resolved sinks.

- **Sources.** Servlet and Spring request accessors, JAX-RS parameters, Flask/Django/FastAPI request objects, Express `req`, PHP superglobals and PSR-7 request methods. Second-order sources are database reads and cache reads whose stored values are tainted elsewhere in the program; LogInjector supports them by treating writes to persistent stores as tainted and matching reads by table or key.
- **Propagation.** Standard string propagation through concatenation, `StringBuilder`, formatting, collections and object fields, with a k-limited call-site sensitivity (k = 1 by default) and field sensitivity for request DTOs.
- **Neutralizer summaries.** Each candidate sanitizer (a method whose input is tainted and whose output reaches a sink) is summarized by the set of contexts it neutralizes. Library sanitizers (OWASP Java Encoder, Apache Commons Text escapers, `html.escape`, `json.dumps`) have hand-written summaries. User-written sanitizers are summarized automatically: LogInjector extracts the string operations on the path (`replace`, `replaceAll`, regex `sub`, character filters) and

checks, with an automaton-based string analysis in the style of Saner [8], whether any context-special character can survive.

- **Reporting rule.** A flow is a candidate when its accumulated neutralizer set does not include ctx(k), following Definition 4. Each candidate carries its source, sink, path, context and the injection classes (C1–C6) the context admits.

5.4 Stage 4 — Dynamic confirmation

Static analysis over-approximates. The dynamic stage removes infeasible and already-neutralized flows.

1. **Harness.** The application runs in a container with its real logging configuration, plus an extra capture appender that records the exact bytes each appender writes. Outbound network access is blocked except for a local canary listener.
2. **Request synthesis.** From the candidate's source description and the application's route table (Spring mappings, Flask routes, OpenAPI specs when present), LogInjector builds an HTTP request that reaches the source. When a path needs authentication, a test account from the project's fixtures is used.
3. **Context-aware payloads.** For each admitted class, a payload family is generated with a unique marker: CRLF plus a forged record matching the layout's pattern for C1; ANSI and backspace sequences for C2; field-breaking quotes for C3; an HTML marker element for C4; a formula with a marker for C5; and a benign lookup that resolves to a canary host or a known string for C6 (never a real exploit gadget).
4. **Oracles.** Each class has a format oracle that parses the captured log with the context grammar G_c. The C1 oracle checks whether the marker appears as its own record; C3 checks whether a new field or a broken parse appears; C4 renders the log in the project's viewer template or a reference viewer in a headless browser and checks whether the marker element exists in the DOM; C6 checks the canary listener or the expanded marker string.
5. **Verdict.** A candidate is *confirmed* if any oracle fires, *refuted* if the request reached the sink (verified by a benign marker) but no oracle fired, and *unconfirmed* if the sink was never reached. Unconfirmed findings are reported with lower priority rather than dropped, so coverage gaps in request synthesis do not silently hide true positives.

5.5 Reports and remediation hints

Each confirmed finding includes the source-to-sink path, the context, the payload and oracle evidence, and a remediation hint ranked by scope: fix at the layout (`%enc{%m}{CRLF}`, JSON encoder), fix at the framework (upgrade, disable lookups), or

fix at the call site (use a parameterized call with a neutralizing wrapper). Layout-level fixes are preferred because they cover every call site at once.

Chapter 6 — Implementation

The prototype targets Java first, because Java has the richest logging ecosystem and the highest-impact interpretation flaw (Log4Shell), and then Python, Node.js and PHP. Adjust this chapter to match what you actually built; the structure below is one workable design.

6.1 Architecture and components

Component	Responsibility	Suggested technology
Config analyzer	Parse manifests and logging configs into the logger context map	Java (Maven model APIs, XML parsing), Python for non-Java configs
Sink specification	Declarative per-framework sink, argument-role and interpretation rules	YAML files, one per framework and version range
Taint engine	Interprocedural taint tracking with neutralizer summaries	CodeQL data-flow library (Java, Python, JS) with custom sink and sanitizer classes, or Soot/SootUp with a custom IFDS client
String analyzer	Summarize user-written sanitizers per context	Automaton library (e.g. dk.brics.automaton)
Harness	Containerized execution with capture appender and canary listener	Docker, a custom Log4j 2 / Logback appender, a local DNS/LDAP/HTTP canary
Request synthesizer	Build requests that reach a source	Route extraction from framework annotations, OpenAPI when available
Oracles	Parse captured logs per context and check markers	Pattern-layout parser, JSON parser, headless browser for HTML viewers
Reporter	SARIF output with evidence and remediation hints	SARIF 2.1.0 for IDE and CI integration

Table 6.1 — LogInjector components

6.2 Sink specification format

Sink rules are data, not code, so new frameworks can be added without changing the engine. An illustrative entry:

```
        framework: log4j-core
        versions: "[2.0, 2.15.0)"
        sinks:
          - method: org.apache.logging.log4j.Logger.
{trace,debug,info,warn,error,fatal}
            args:
              - position: 0
                role: template
                interpretation: [placeholder, lookup]
              - position: 1..
                role: parameter
                interpretation: [lookup]
        context_from: logger_config
```

A newer version range of the same framework declares `interpretation: [placeholder]` only, so the C6 class is not admitted and no lookup payloads are generated.

6.3 Capture appender

The harness inserts an appender alongside each configured appender, using the same layout instance, so the captured bytes are exactly what the real appender writes. Each captured record is tagged with the request identifier of the synthesized request (through MDC), which lets an oracle attribute records to candidates even when many requests run in parallel.

6.4 Safety of dynamic confirmation

The harness runs with no external network access. C6 payloads only resolve lookups to a local canary or to a harmless built-in lookup (for example, a lookup that expands to a fixed marker string), and never load remote code. Confirmation runs only against applications under the analyst’s control. These controls are required for the tool to be ethically usable and are discussed in Chapter 8.

6.5 Size and availability

[Report lines of code per component, the number of sink rules and library sanitizer summaries, and the repository link and license for artifact evaluation.]

Chapter 7 — Evaluation

This chapter defines the methodology for each research question. All numbers are placeholders in square brackets; fill them from your experiments.

7.1 Subjects

Real-world corpus. Select [N] open-source web applications by these criteria: actively maintained, at least [X] GitHub stars, a web entry point (Spring, Jakarta EE, Flask, Django, Express, Laravel or Symfony), and one of the supported logging frameworks. Record each project's size, framework and logging configuration.

Application	Language	Web framework	Logging framework	KLOC	Log call sites
[App 1]	Java	Spring Boot	Logback	[ ]	[ ]
[App 2]	Java	Jakarta EE	Log4j 2	[ ]	[ ]
[App 3]	Python	Django	logging	[ ]	[ ]
[App 4]	JavaScript	Express	Winston	[ ]	[ ]
[App 5]	PHP	Laravel	Monolog	[ ]	[ ]

Table 7.1 — Evaluation subjects

Seeded benchmark (LogInjBench). Build a benchmark of [M] small test cases, each a vulnerable or safe variant, covering classes C1–C6, each supported framework, and each neutralization style (layout encoding, structured encoder, call-site sanitizer, incomplete sanitizer, framework configuration). Include tricky safe cases, such as a CRLF-stripping sanitizer before a plain-text sink, and tricky vulnerable cases, such as the same sanitizer before an HTML viewer. The OWASP Benchmark project's test-case style is a useful model.

7.2 Baselines

Compare against: CodeQL's standard log-injection queries; Semgrep log-injection rules; a commercial or open-source DAST scanner (for example OWASP ZAP) configured to scan the same applications; and two ablated variants of LogInjector (Section 7.5).

7.3 Ground truth

For the benchmark, ground truth is known by construction. For real applications, two authors independently label every report from any tool as true or false positive, resolving disagreements by discussion and recording Cohen's kappa [κ =]. Recall

on real applications is estimated against the union of confirmed true positives across all tools, which is a lower bound on the true vulnerability count; this limitation is noted in Chapter 8.

7.4 RQ1 and RQ2 — Prevalence and effectiveness

Tool	TP	FP	FN	Precision	Recall	F1
CodeQL (log-injection queries)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Semgrep rules	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
DAST scanner	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
LogInjector	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 7.2 — Detection effectiveness

Report prevalence by class: the share of applications with at least one confirmed flow per class, and the distribution of sources (headers, parameters, bodies, second-order). Report results separately for the benchmark and the real corpus.

7.5 RQ3 — Ablation

Variant	Context model	Dynamic confirmation	Precision	Recall
Taint only	No	No	[ ]	[ ]
Taint + context	Yes	No	[ ]	[ ]
Taint + confirmation	No	Yes	[ ]	[ ]
Full LogInjector	Yes	Yes	[ ]	[ ]

Table 7.3 — Ablation study

The hypothesis is that context modeling removes false positives where a layout-level neutralizer exists, and confirmation removes infeasible paths. Analyze any recall lost by confirmation (true positives the harness failed to reach).

7.6 RQ4 — Cost

Report wall-clock time and peak memory per stage and per application, on a stated machine ([CPU, cores, RAM]). Relate analysis time to KLOC. State whether the static stages fit a typical CI time budget of [X] minutes and whether confirmation is better run nightly.

7.7 RQ5 — Real-world impact

List the previously unknown vulnerabilities found, their class, and disclosure status. Follow coordinated disclosure: report privately, allow [90] days, and record whether maintainers confirmed, fixed, or assigned a CVE.

ID	Application	Class	Source	Status
[V1]	[ ]	[C1–C6]	[ ]	[Reported / Confirmed / Fixed / CVE]

Table 7.4 — New vulnerabilities

7.8 Case studies

Describe two or three findings in depth. Good candidates: a CRLF forging flaw in an authentication handler that lets an attacker forge successful-login records; a stored XSS through `User-Agent` in an admin audit-log page; and a flow that generic tools flagged but LogInjector correctly dismissed because a JSON encoder neutralized it.

Chapter 8 — Discussion, Limitations and Ethics

8.1 Why context matters

The central lesson is that “tainted data reaches a log call” is not a vulnerability by itself. Whether it is depends on the layout, the framework version and the consumer. Tools that ignore context must choose between noise and silence. Modeling the context turns log injection from an unactionable warning class into a precise one, and points developers to the cheapest fix: one layout change often neutralizes hundreds of call sites.

8.2 Limitations

- **Configuration resolution.** Logging configured programmatically, through environment variables, or at deployment time (for example, in a Kubernetes ConfigMap) may not be visible to the analyzer. LogInjector falls back to framework defaults, which can produce false positives or negatives.
- **Downstream consumers.** The C4 and C5 classes depend on viewers outside the application. LogInjector models consumers the analyst declares and a reference viewer; it cannot know every tool an organization uses.
- **Dynamic coverage.** Request synthesis may fail to reach deep paths that need complex state, multi-step workflows or valid business data. These findings remain unconfirmed rather than refuted.
- **Language coverage.** The prototype’s depth varies by language; reflection, dynamic dispatch in JavaScript, and PHP’s dynamic features reduce static precision.
- **Second-order flows.** Matching store writes to reads by table or key is approximate.

8.3 Threats to validity

Internal. Manual labeling may contain errors; two independent labelers and an agreement statistic mitigate this. **External.** The selected applications may not represent closed-source enterprise software, which often has more complex logging pipelines. **Construct.** Recall on real applications is measured against the union of findings across tools, which underestimates the true number of vulnerabilities. Benchmark cases written by the authors may favor the tool’s design; mitigate by including cases written before the tool was finalized, or by external contributors.

8.4 Ethics and responsible disclosure

The dynamic stage executes attack payloads, so it must only be run against systems the analyst owns or is authorized to test. Interpretation payloads (C6) are restricted to local canaries and harmless lookups and never deliver remote code. All new vulnerabilities were [are to be] reported privately to maintainers under coordinated disclosure before publication. If your institution requires ethics or research-integrity review for vulnerability research, record the approval here.

8.5 Recommendations for practitioners

1. Prefer structured logging with an encoder that escapes control characters and quotes.
2. Add layout-level encoding (such as `%enc{%m}{CRLF}`) for text layouts.
3. Keep logging frameworks patched and disable message interpretation features that are not needed.
4. Encode log content on output in every viewer and dashboard; treat logs as untrusted input.
5. Prefix cells that start with `=`, `+`, `-` or `@` when exporting logs to spreadsheets.

Chapter 9 — Conclusion and Future Work

This dissertation argued that log injection is a context-dependent vulnerability class that existing tools handle poorly, and presented LogInjector to detect it. LogInjector combines a model of logging frameworks' rendering contexts with context-sensitive static taint analysis and format-aware dynamic confirmation. The formal model defines a vulnerability as a tainted flow that is not neutralized *for its context*, which explains why the same code can be safe in one deployment and vulnerable in another. The evaluation [summarize the main results for RQ1–RQ5 in two or three sentences].

Future work.

- **Configuration-as-code.** Extend Stage 1 to read deployment manifests, Helm charts and environment configuration, closing the largest source of context uncertainty.
- **Pipeline-wide analysis.** Model the full path from appender to SIEM, including Logstash and Fluentd parsing rules, to detect injections that only bite after re-parsing.
- **Automatic repair.** Generate layout-level patches (encoder changes) and verify them with the same oracles.
- **LLM-assisted sanitizer summarization.** Use language models to propose summaries for unusual sanitizers, verified by the string analyzer so that the model's output is never trusted alone.
- **Log integrity.** Combine detection with tamper-evident logging (hash chaining, signed records) so that forged entries are both prevented and detectable.

References

These are well-known works cited from memory, not looked up for this draft. Check each title, venue and year against the original before submission, and add page numbers and DOIs in your institution’s style. The same list is supplied as BibTeX and RIS files.

1. V. B. Livshits and M. S. Lam. Finding Security Vulnerabilities in Java Applications with Static Analysis. USENIX Security Symposium, 2005.
2. N. Jovanovic, C. Kruegel and E. Kirda. Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities. IEEE Symposium on Security and Privacy, 2006.
3. O. Tripp, M. Pistoia, S. J. Fink, M. Sridharan and O. Weisman. TAJ: Effective Taint Analysis of Web Applications. PLDI, 2009.
4. S. Arzt et al. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. PLDI, 2014.
5. F. Spoto et al. Static Identification of Injection Attacks in Java. ACM TOPLAS, 2019.
6. F. Yamaguchi, N. Golde, D. Arp and K. Rieck. Modeling and Discovering Vulnerabilities with Code Property Graphs. IEEE Symposium on Security and Privacy, 2014.
7. P. Avgustinov, O. de Moor, M. P. Jones and M. Schäfer. QL: Object-oriented Queries on Relational Data. ECOOP, 2016.
8. D. Balzarotti et al. Saner: Composing Static and Dynamic Analysis to Validate Sanitization in Web Applications. IEEE Symposium on Security and Privacy, 2008.
9. M. Samuel, P. Saxena and D. Song. Context-Sensitive Auto-Sanitization in Web Templating Languages Using Type Qualifiers. ACM CCS, 2011.
10. P. Saxena, D. Molnar and B. Livshits. ScriptGard: Automatic Context-Sensitive Sanitization for Large-Scale Legacy Web Applications. ACM CCS, 2011.
11. Z. Su and G. Wassermann. The Essence of Command Injection Attacks in Web Applications. POPL, 2006.
12. A. Doupé, M. Cova and G. Vigna. Why Johnny Can’t Pentest: An Analysis of Black-box Web Vulnerability Scanners. DIMVA, 2010.
13. J. Bau, E. Bursztein, D. Gupta and J. Mitchell. State of the Art: Automated Black-Box Web Application Vulnerability Testing. IEEE Symposium on Security and Privacy, 2010.
14. A. Kieyzun, P. J. Guo, K. Jayaraman and M. D. Ernst. Automatic Creation of SQL Injection and Cross-Site Scripting Attacks. ICSE, 2009.

15. E. J. Schwartz, T. Avgerinos and D. Brumley. All You Ever Wanted to Know About Dynamic Taint Analysis and Forward Symbolic Execution (but Might Have Been Afraid to Ask). IEEE Symposium on Security and Privacy, 2010.
16. P. He, J. Zhu, Z. Zheng and M. R. Lyu. Drain: An Online Log Parsing Approach with Fixed Depth Tree. IEEE ICWS, 2017.
17. J. Zhu et al. Tools and Benchmarks for Automated Log Parsing. ICSE-SEIP, 2019.
18. MITRE. CWE-117: Improper Output Neutralization for Logs. Common Weakness Enumeration.
19. OWASP Foundation. Logging Cheat Sheet. OWASP Cheat Sheet Series.
20. Apache Software Foundation. Apache Log4j Security Vulnerabilities (CVE-2021-44228, CVE-2021-45046 and related advisories).

Appendix A — Payload families per class

Class	Payload template	Oracle check
C1	`<marker>\r\n<forged record in layout pattern>`	Marker parsed as a separate record
C2	`<marker>` + ANSI erase / backspace sequences	Raw bytes contain control sequences after the marker
C3	`<marker>","<field>":"<value>` (and key-value equivalents)	New field appears, or record fails to parse
C4	`<span id="<marker>">`	Marker element present in rendered viewer DOM
C5	`="<marker>"`	Exported cell begins with a formula character, unescaped
C6	Benign lookup resolving to marker or local canary	Expanded marker in log, or canary hit

Appendix B — Supported sources and sinks

[Insert the full list of source APIs per web framework and sink methods per logging framework from the sink specification files.]

Appendix C — Artifact

[Repository URL, license, build instructions, scripts to reproduce every table in Chapter 7, and the LogInjBench benchmark.]